Bazar.com - Design Report

Distributed and Operating Systems - Fall 2026 - Lab 1

1. Overview and Architecture

Bazar.com is a small online bookstore offering four books in two topics: distributed systems and undergraduate school. The system uses two logical tiers and three independently running Flask microservices. The frontend tier contains Frontend; the backend tier contains Catalog and Order. Each service runs in its own Linux Docker container.

Frontend exposes search, info, and purchase operations to clients. Search and info requests are forwarded to Catalog. Purchase requests are forwarded to Order, which communicates with Catalog before recording a successful order. All communication uses HTTP REST with JSON responses. No service directly accesses another service's database.

Service addresses are configurable through environment variables. Docker Compose supplies internal addresses using service names: catalog:5001 and order:5002. This separates service discovery from application logic and allows deployment on different machines by changing the configured URLs. The host client connects to Frontend on port 5000.

2. Catalog and Persistent Storage

Catalog owns book ids, titles, topics, quantities, and prices. It provides GET /search/<topic>, GET /info/<id>, and PATCH /update/<id>. Search returns matching ids and titles; info returns complete book details. Update accepts a price, an inventory adjustment, or both.

Catalog uses SQLite rather than a separate database server. Prices are stored as integer minor units to avoid floating-point arithmetic errors during storage and calculations. SQL statements use bound parameters. Initial books are inserted without resetting existing records when the service restarts.

Order owns a separate SQLite database containing successful purchases. Each record stores an order id, book id, title, quantity, price, and UTC creation time. Storing the purchase title and price preserves a historical snapshot if catalog information changes later.

Docker named volumes preserve both databases when containers are removed and recreated. Frontend is stateless. Local execution outside Docker uses separate database files.

3. Purchase Processing and Concurrency

A purchase requests exactly one copy. Order first queries Catalog to verify that the book exists and appears to be in stock. It then requests an inventory decrement. Because another request may purchase the last copy between these calls, the initial query is advisory; the atomic decrement result determines whether the purchase proceeds.

Catalog starts an update transaction with BEGIN IMMEDIATE, checks existence and inventory, applies the changes, and reads the updated book before committing. Stock cannot become negative. Combined price and quantity updates are rejected together if the requested inventory change is impossible.

Order uses the price returned by the successful decrement, then commits the order to its own database. Only after that commit does it print "bought book" followed by the title and return HTTP 201. Failed purchases are not recorded as successful orders.

SQLite connections are created separately for requests, use a five-second lock timeout, and operate with WAL mode. These choices support concurrent requests while keeping transactions short. SQLite still serializes writes, limiting throughput under heavy load.

4. Interfaces and Failure Handling

Frontend forwards valid backend responses while preserving their HTTP status codes and JSON structure. Missing books return 404; insufficient inventory returns 409. Invalid input returns 400. Network failures return 503, timeouts return 504, and invalid backend responses return 502.

Services use bounded HTTP waits. Purchase requests are forwarded once without automatic retries, because a lost reply may hide a completed purchase. Internal diagnostics are logged without exposing database details or stack traces to clients.

The command-line client supports search, info, and purchase. It displays search results and book details clearly and prints the purchase message on success. The client runs on the host operating system while the three services run in containers.

5. Testing and Observed Results

Catalog has 21 passing HTTP integration tests. Order has 13 passing tests using a mock Catalog and a temporary database. Frontend has 14 passing tests using mocked backend calls. The tests cover successful requests, invalid input, missing books, stock conflicts, network failures, error propagation, purchase price selection, and printing the success message after the order commit.

Live Docker checks verified successful purchases, unchanged order counts after rejected requests, and persistence after container recreation. Stopping Catalog caused Order to return 503 without creating an order.

A full-system concurrency check sent ten purchases through Frontend with one copy available. Exactly one returned 201, nine returned 409, inventory reached zero during the test, and exactly one order was added. Client output, service logs, test output, and screenshots are included in docs/.

6. Running the Program

From the repository root, with Docker and Compose available:

```
docker compose up -d --build
docker compose ps
```

Allow services to initialize. Prepare the host client:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r client/requirements.txt
python client/client.py search "distributed systems"
python client/client.py info 2
python client/client.py purchase 2
```

Each purchase command changes persistent data. Inspect component output with docker compose logs catalog order frontend. Stop the system with docker compose down. Adding -v deletes the databases and should only be used for an intentional reset.

For tests, install requests and pytest, then run the test commands listed in README.md. Inventory-modifying checks require an isolated test instance without other writers.

7. Tradeoffs, Known Limitations, and Improvements

Flask and SQLite keep the implementation small and straightforward. Separate databases preserve service ownership, but Catalog and Order do not share a distributed transaction. If stock decrement succeeds and Order fails before saving the order, stock may decrease without a matching order record. If the response is lost, the client may not know whether the purchase completed. Retrying can create another purchase because the current API does not provide idempotency.

A future version could introduce stable purchase identifiers, idempotent reservation and confirmation operations, and reconciliation of incomplete purchases. This would require coordinated changes to both backend services.

Other improvements include stronger validation of successful backend response schemas, service health checks for startup coordination, and a production WSGI server. Authentication and transport security would also be needed for deployment beyond the local lab environment.